

Lab Manual 7: Implementation of the Concept of Polymorphism

Objective:

Understand and implement the concept of **Polymorphism** in Object-Oriented Programming (OOP).
Learn how to use **Method Overloading** (Compile-time Polymorphism) and **Method Overriding** (Runtime Polymorphism) with easy, real-life scenarios

Theory:

What is Polymorphism?

Polymorphism means "many forms." In programming, it allows you to use the same method name but with different behavior.

There are two types of Polymorphism:

1. Compile-time Polymorphism (Static Polymorphism):

Achieved through **Method Overloading**.

The method to be called is determined during the **compilation** phase.

Method Overloading occurs when a class has multiple methods with the same name but different parameters (e.g., different number of parameters or parameter types).

2. Runtime Polymorphism (Dynamic Polymorphism):

Achieved through **Method Overriding**.

The method to be called is determined during **runtime**, i.e., at the time the program is executing.

Method Overriding occurs when a subclass provides a specific implementation for a method that is already defined in its parent class.

Syntax:

1. Method Overloading (Compile-time Polymorphism):

```
class ClassName {  
    // Method with different parameter types or number of  
    parameters  
    returnType methodName(type1 param1) { ... }  
    returnType methodName(type2 param1, type2 param2) { ... } }
```

Line-by-Line Explanation:

1. **class ClassName:** This line defines a class named `ClassName`.

2. **returnType methodName(type1 param1) { ... }**

returnType: This specifies the type of value that the method will return.

Example: If you want the method to return a number, you can use `int` or `double`. If it doesn't return anything, you can use `void`.

methodName: This is the name of the method. You can call this method by its name.

Example: It could be `addNumbers` or `printMessage`.

type1 param1: This part defines a **parameter** the method will take. The parameter type is `type1` (e.g., `int`, `String`), and it's named `param1`.

Example: If you pass an integer to the method, `type1` would be `int`, and `param1` would represent that integer.

3. **returnType methodName(type2 param1, type2 param2) { ... }**

This is another method with the same **name** (`methodName`), but it has **two parameters**.

type2 param1 and type2 param2: These are the two parameters that the method will take. Both are of type `type2` (e.g., `int`, `String`).

Example: If you are passing two integers to the method, `type2` would be `int`, and the two parameters would represent those integers.

Examples and Scenarios:

Scenario 1: Method Overloading (Compile-time Polymorphism)

Real-life Scenario:

In a university's mini calculator app, students can perform basic operations like adding two numbers, three numbers, or even decimal numbers. Create a `Calculator` class using method overloading to handle all these operations. Demonstrate method overloading using different parameter types and number of parameters

Code Example (Method Overloading):

```

class Calculator {
    // Method to add two integers
    int add(int a, int b) {
        return a + b;
    }
    // Overloaded method to add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }
    // Overloaded method to add two decimal numbers
    double add(double a, double b) {
        return a + b;
    }
}

public class CalculatorExample {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Sum of 2 integers: " + calc.add(5, 10));
        System.out.println("Sum of 3 integers: " + calc.add(5, 10, 15));
        System.out.println("Sum of 2 decimals: " + calc.add(5.5, 10.5));
    }
}

```

Question: How is this method overloading?

Answer : This is **method overloading** because:

Same method name: All the methods are named **add**.

Different parameters: Each method has a **different number or type of parameters** (2 integers, 3 integers, 2 doubles).

Scenario 2:

Your university wants to develop a **Smart Calculator** that performs basic operations like **Addition, Subtraction, Multiplication, and Division**.

However, students may enter either **integers** or **decimal values**, and sometimes may want to operate on **2 or 3 numbers at a time**. You have to write a program using **Method Overloading** to handle all these different types of inputs using the **same method name**.

Requirements:

1. Use **Method Overloading** for each operation (add, subtract, multiply, divide).
2. Create versions of each method:

For **2 integers**

For **3 integers**

For **2 double values**

3. Implement all methods under one class.

Code:

```
// SmartCalculator class jo method overloading ka use kar raha hai
class SmartCalculator {

    // 2 integer values ka addition
    int add(int a, int b) {
        return a + b;
    }

    // 3 integer values ka addition
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // 2 decimal (double) values ka addition
    double add(double a, double b) {
        return a + b;
    }

    // 2 integer values ka subtraction
    int subtract(int a, int b) {
        return a - b;
    }

    // 3 integer values ka subtraction
    int subtract(int a, int b, int c) {
        return a - b - c;
    }

    // 2 decimal values ka subtraction
    double subtract(double a, double b) {
        return a - b;
    }
}
```

```

    }
    // 2 integer values ka multiplication
    int multiply(int a, int b) {
        return a * b;
    }
    // 3 integer values ka multiplication
    int multiply(int a, int b, int c) {
        return a * b * c;
    }
    // 2 decimal values ka multiplication
    double multiply(double a, double b) {
        return a * b;
    }
    // 2 integer values ka division
    int divide(int a, int b) {
        return a / b; // integer division
    }
    // 2 decimal values ka division
    double divide(double a, double b) {
        return a / b;
    }
}

// Main class jahan hum objects banake methods call karenge
public class CalculatorTest {
    public static void main(String[] args) {
        // SmartCalculator ka object banaya
        SmartCalculator calc = new SmartCalculator();
        // Different type ke overloaded methods call ho rahe hain
        System.out.println("Addition of 2 ints: " + calc.add(5, 10));
        System.out.println("Addition of 3 ints: " + calc.add(5, 10, 15));
        System.out.println("Addition of 2 doubles: " + calc.add(5.5, 10.5));
        System.out.println("Subtraction of 2 ints: " + calc.subtract(20, 5));
        System.out.println("Subtraction of 3 ints: " + calc.subtract(30, 10, 5));
        System.out.println("Subtraction of 2 doubles: " + calc.subtract(20.5, 5.5));
        System.out.println("Multiplication of 2 ints: " + calc.multiply(3, 4));
        System.out.println("Multiplication of 3 ints: " + calc.multiply(2, 3, 4));
        System.out.println("Multiplication of 2 doubles: " + calc.multiply(3.5, 2.5));
        System.out.println("Division of 2 ints: " + calc.divide(10, 2));
        System.out.println("Division of 2 doubles: " + calc.divide(10.5, 2.5));
    }
}

```

Method Overriding (Runtime Polymorphism):

Defination:

Method Overriding is a fundamental concept in **Polymorphism**, and it allows a subclass to provide a specific implementation of a method that is

already defined in its superclass. This is particularly useful when you want to change or extend the behavior of an inherited method.

Here's a breakdown of important points about method overriding:

1. Purpose of Method Overriding

The main purpose of method overriding is to change or extend the functionality of an inherited method in a subclass. It allows different objects of different subclasses to call the same method but exhibit different behaviors.

2. How Method Overriding Works

When a subclass provides its own implementation of a method that is already defined in the superclass, it overrides the method. The method signature (name, return type, and parameters) must be the same as in the superclass.

3. Runtime Polymorphism

Method overriding is a way to achieve **runtime polymorphism**. It means that Java decides which method to call at runtime based on the object type (whether it's a subclass or superclass object). This allows objects of different subclasses to invoke the overridden method, resulting in different behaviors depending on the actual class of the object.

4. Rules for Method Overriding

Method Signature: The overridden method in the subclass must have the same method signature (name, return type, and parameters) as the method in the superclass.

Access Modifier: The overriding method should have the same or a more permissive access level than the method in the superclass. For example, if the superclass method is `public`, the subclass method must also be `public`.

Return Type: The return type of the overridden method in the subclass must be the same or a subtype of the return type in the superclass (called **covariant return type**).

No Static Methods: You cannot override static methods. Static methods are resolved at compile-time, not at runtime.

Method overriding is done using the **@Override** annotation in Java, though the annotation itself is not strictly required. It is recommended to use it as a best practice because it helps catch errors at compile time.

No specific keyword is required to override a method, but the **@Override** annotation is used to indicate that you are overriding a method from the superclass.

@Override ensures that:

1. The method you're overriding exists in the superclass.
2. The method signature (method name, parameters, return type) is exactly the same as in the superclass.

Syntax:

```
class Parent {  
    void methodName() {  
        // Parent class method  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void methodName() {  
        // Child class method  
    }  
}
```

Scenario 3: Method Overriding (Runtime Polymorphism)

Imagine you're at a **Food Delivery Service**. There is a general **Food** class that describes any kind of food. However, you have **Pizza** and **Burger** classes that need to provide specific details about their preparation. Even though the method `prepareFood()` exists in the **Food** class, each type of food will have its own implementation of how it's prepared.

Requirements :

1. Class Structure:

Food Class: A general class representing any food, containing a method `prepareFood()`.

Pizza Class: A subclass of Food, overriding `prepareFood()` to specify how pizza is prepared.

Burger Class: A subclass of Food, overriding `prepareFood()` to specify how a burger is prepared.

2. Method Overriding:

In the **Pizza** and **Burger** classes, override the `prepareFood()` method to provide specific behavior (e.g., pizza is baked, burger is grilled).

3. Main Class:

In the **main** method, create instances of **Pizza** and **Burger** using the **Food** class reference.

Call the **prepareFood()** method on both instances to demonstrate polymorphism.

Code:

```
class Food {
    // Parent class method
    void prepareFood() {
        System.out.println("Food is being prepared.");
    }
}

class Pizza extends Food {
    // Overridden method for Pizza
    @Override
    void prepareFood() {
        System.out.println("Pizza is being baked.");
    }
}

class Burger extends Food {
    // Overridden method for Burger
    @Override
    void prepareFood() {
        System.out.println("Burger is being grilled.");
    }
}

public class FoodPreparation {
    public static void main(String[] args) {
        // Creating objects of child classes
        Food food1 = new Pizza(); // Runtime Polymorphism
        Food food2 = new Burger(); // Runtime Polymorphism
        // Calling the overridden methods
        food1.prepareFood(); // Output: Pizza is being baked.
        food2.prepareFood(); // Output: Burger is being grilled.
    }
}
```

When to Use Constructors in Polymorphism?

Method Overloading: Constructors are not typically used when overloading methods. Overloading deals with multiple methods with the same name but different parameters. Constructors are used for initializing an object when it is created.

Method Overriding: Constructors are used in the child class to initialize the object. The child class constructor can call the parent class constructor using **super()** to ensure proper initialization.